Run samples of the ServiceQueue simulation Natalie Huey 2

Collect statistics and plot histograms along the way.

Set up

We'll measure time in hours

Arrival rate: every 75 seconds

```
lambda = 0.8;
```

Departure (service) rate: 6.5 minutes

```
mu = 1/6.5;
```

Number of serving stations:

```
for s=1:8
```

Run 100 samples of the queue.

```
NumSamples = 100;
```

Each sample is run up to a maximum time.

```
MaxTime = 240; %four hours in minutes
```

Make a log entry every so often

```
LogInterval = 1;
```

Numbers from theory for M/M/k queue

Compute `P(1+n)` = P_n = probability of finding the system in state n in the long term.

```
rho = lambda / (s*mu);
a = lambda / mu;
sum1=0;
nMax = 10;
for n=0:(s-1)
    sum1= sum1+a^n/factorial(n);
end
sum2=(a^s / factorial(s))*(1 / (1-rho));
P0 = 1 / (sum1 + sum2);
P= zeros(1,nMax+1);
for n=0:nMax
    if n<s
```

```
        P(n+1) = (a^n / factorial(n))*P0;
    else
        P(n+1) = (a^n / (factorial(s) * s^(n-s)))*P0;
    end
end
```

Run simulation samples

This is the most time consuming calculation in the script, so let's put it in its own section. That way, we can run it once, and more easily run the faster calculations multiple times as we add features to this script.

Reset the random number generator. This causes MATLAB to use the same sequence of pseudo-random numbers each time you run the script, which means the results come out exactly the same. This is a good idea for testing purposes. Under other circumstances, you probably want the random numbers to be truly unpredictable and you wouldn't do this.

```
rng("default");
```

We'll store our queue simulation objects in this list.

```
QSamples = cell([NumSamples, 1]);
```

The statistics come out weird if the log interval is too short, because the log entries are not independent enough. So the log interval should be long enough for several arrival and departure events happen.

```
for SampleNum = 1:NumSamples
    %fprintf("Working on sample %d\n", SampleNum);
    q = ServiceQueue( ...
        ArrivalRate=lambda, ...
        DepartureRate=mu, ...
        NumServers=s, ...
        LogInterval=LogInterval);
    q.schedule_event(Arrival(random(q.InterArrivalDist), Customer(1)));
    run_until(q, MaxTime);
    QSamples{SampleNum} = q;
end
```

Collect measurements of how many customers are in the system

Count how many customers are in the system at each log entry for each sample run. There are two ways to do this. You only have to do one of them.

Option one: Use a for loop.

```
NumInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    % Pull out samples of the number of customers in the queue system. Each
    % sample run of the queue results in a column of samples of customer
    % counts, because tables like q.Log allow easy extraction of whole
```

```
    % columns like this.
    NumInSystemSamples{SampleNum} = q.Log.NumWaiting + q.Log.NumInService;
end
```

Option two: Map a function over the cell array of ServiceQueue objects.

The `@(q) ...` expression is shorthand for a function that takes a `ServiceQueue` as input, names it `q`, and computes the sum of two columns from its log. The `cellfun` function applies that function to each item in `QSamples`. The option `UniformOutput=false` tells `cellfun` to produce a cell array rather than a numerical array.

```
NumInSystemSamples = cellfun( ...
    @(q) q.Log.NumWaiting + q.Log.NumInService, ...
    QSamples, ...
    UniformOutput=false);
```

Join numbers from all sample runs.

`vertcat` is short for "vertical concatenate", meaning it joins a bunch of arrays vertically, which in this case results in one tall column.

```
NumInSystem = vertcat(NumInSystemSamples{:});
```

MATLAB-ism: When you pull multiple items from a cell array, the result is a "comma-separated list" rather than some kind of array. Thus, the above means

`NumInSystem = vertcat(NumInSystemSamples{1}, NumInSystemSamples{2}, ...)`

which concatenates all the columns of numbers in NumInSystemSamples into one long column.

This is roughly equivalent to "splatting" in Python, which looks like `f(*args)`.

Now to calculate the mean number of customers waiting in our simulation:

```
NumWaitingSamples = cellfun( ...
    @(q) q.Log.NumWaiting, ...
    QSamples, ...
    UniformOutput=false);

NumWaiting = vertcat(NumWaitingSamples{:});

Lq = mean(NumWaiting);

fprintf("Simulated L_q: %f\n", Lq);
```

```
Simulated L_q: 78.341639
Simulated L_q: 58.889051
Simulated L_q: 40.740605
Simulated L_q: 24.121492
Simulated L_q: 8.899992
Simulated L_q: 2.982970
Simulated L_q: 0.787751
```

```
Simulated L_q: 0.354232
```

Pictures and stats for number of customers in system

Print out mean number of customers in the system.

```
meanNumInSystem = mean(NumInSystem);
fprintf("Mean number in system: %f\n", meanNumInSystem);
```

Make a figure with one set of axes.

```
fig = figure();
t = tiledlayout(fig,1,1);
ax = nexttile(t);
```

MATLAB-ism: Once you've created a picture, you can use `hold` to cause further plotting functions to work with the same picture rather than create a new one.

```
hold(ax, "on");
```

Start with a histogram. The result is an empirical PDF, that is, the area of the bar at horizontal index n is proportional to the fraction of samples for which there were n customers in the system. The data for this histogram is counts of customers, which must all be whole numbers. The option `BinMethod="integers"` means to use bins $(-0.5, 0.5), (0.5, 1.5), \ldots$ so that the height of the first bar is proportional to the count of 0s in the data, the height of the second bar is proportional to the count of 1s, etc. MATLAB can choose bins automatically, but since we know the data consists of whole numbers, it makes sense to specify this option so we get consistent results.

```
h = histogram(ax, NumInSystem, Normalization="probability",
BinMethod="integers");
```

Plot $(0, P_0), (1, P_1), \ldots$. If all goes well, these dots should land close to the tops of the bars of the histogram.

```
plot(ax, 0:nMax, P, 'o', MarkerEdgeColor='k', MarkerFaceColor='r');
```

Add titles and labels and such.

```
title(ax, "Number of customers in the system");
xlabel(ax, "Count");
ylabel(ax, "Probability");
legend(ax, "simulation", "theory");
```

Set ranges on the axes. MATLAB's plotting functions do this automatically, but when you need to compare two sets of data, it's a good idea to use the same ranges on the two pictures. To start, you can let MATLAB choose the ranges automatically, and just know that it might choose very different ranges for different sets of data. Once you're certain the picture content is correct, choose an x range and a y range that gives good results for

all sets of data. The final choice of ranges is a matter of some trial and error. You generally have to do these commands *after* calling `plot` and `histogram`.

This sets the vertical axis to go from 0 to 0.3.

```
ylim(ax, [0, 0.3]);
```

This sets the horizontal axis to go from -1 to 21. The histogram will use bins $(-0.5, 0.5), (0.5, 1.5), \ldots$ so this leaves some visual breathing room on the left.

```
xlim(ax, [-1, 21]);
```

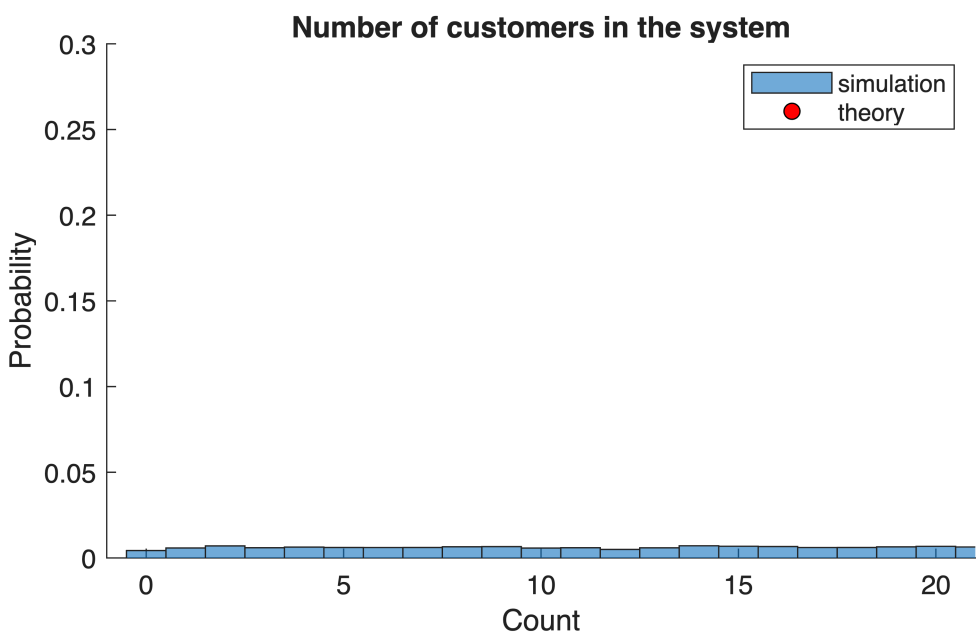
MATLAB-ism: You have to wait a couple of seconds for those settings to take effect or `exportgraphics` will screw up the margins.

```
pause(2);
```

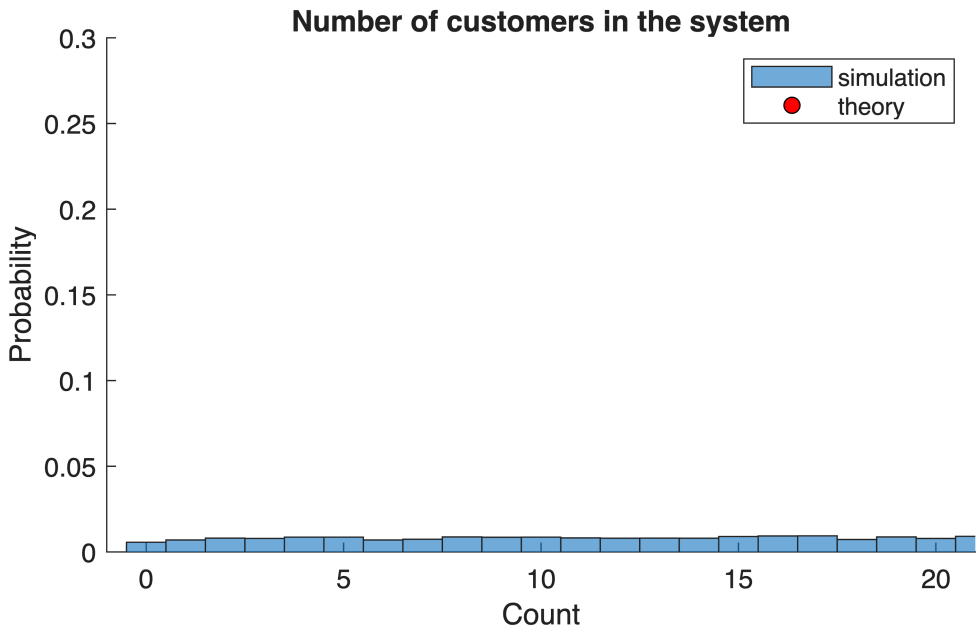
Save the picture as a PDF file.

```
exportgraphics(fig, "Number in system histogram.pdf");
```

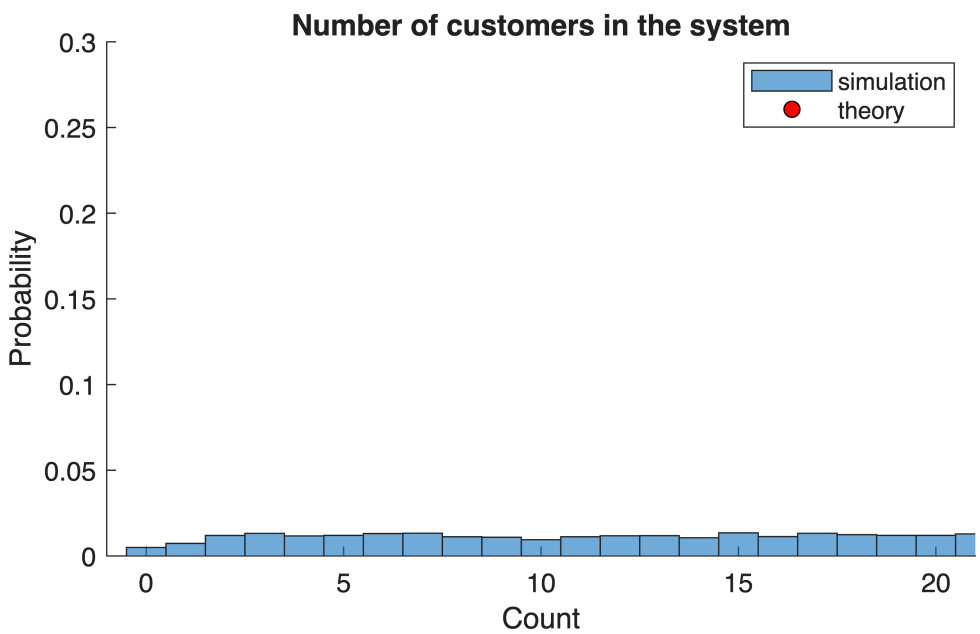
```
Mean number in system: 79.337313
```



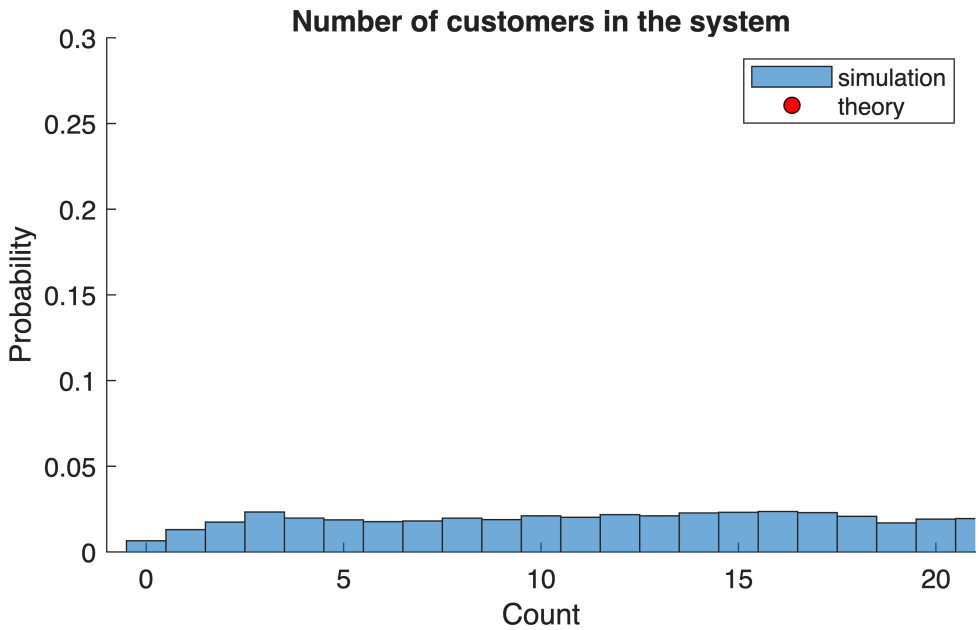

```
Mean number in system: 60.870740
```



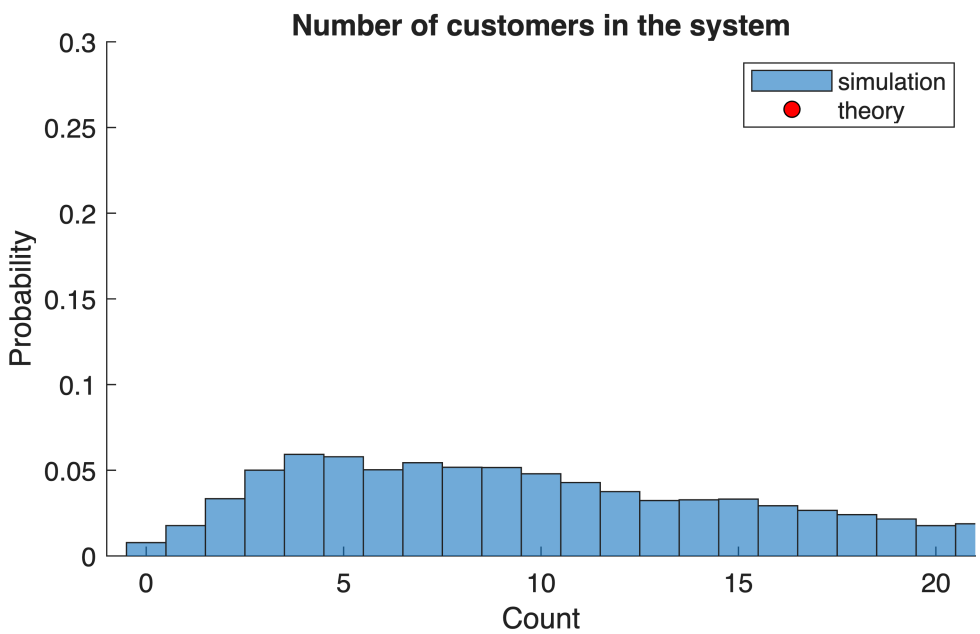

```
Mean number in system: 43.699155
```



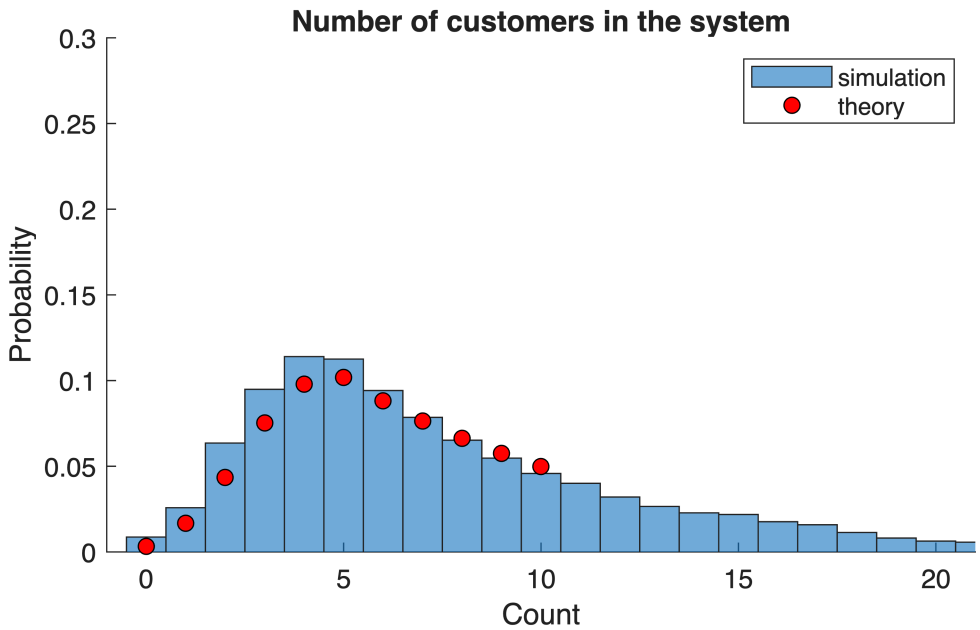

```
Mean number in system: 27.998126
```



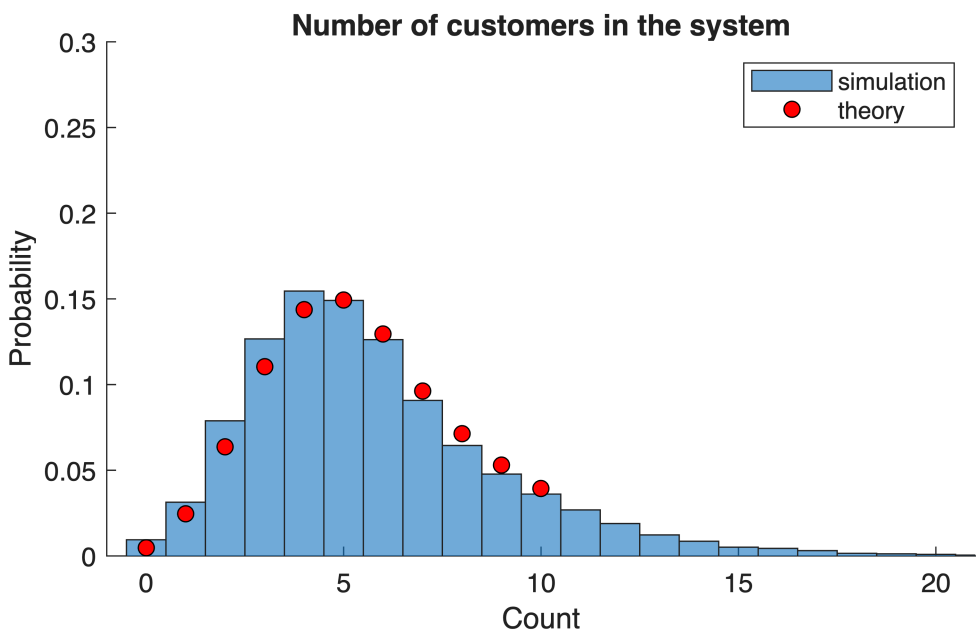

```
Mean number in system: 13.530506
```



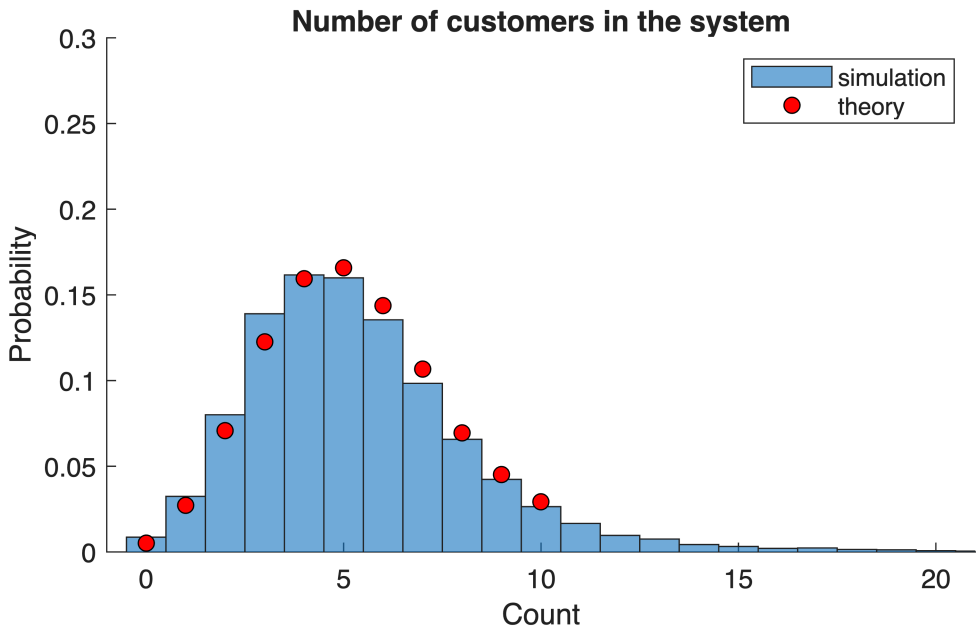

```
Mean number in system: 7.922305
```




```
Mean number in system: 5.744577
```




```
Mean number in system: 5.387203
```



Collect measurements of how long customers spend in the system

This is a rather different calculation because instead of looking at log entries for each sample `ServiceQueue`, we'll look at the list of served customers in each sample `ServiceQueue`.

Option one: Use a for loop.

```
TimeInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    % The next command has many parts.
    %
    % q.Served is a row vector of all customers served in this particular
    % sample.
    % The ' on q.Served' transposes it to a column.
    %
    % The @(c) ... expression below says given a customer c, compute its
    % departure time minus its arrival time, which is how long c spent in
    % the system.
    %
    % cellfun(@(c) ..., q.Served') means to compute the time each customer
    % in q.Served spent in the system, and build a column vector of the
    % results.
    %
    % The column vector is stored in TimeInSystemSamples{SampleNum}.
    TimeInSystemSamples{SampleNum} = ...
        cellfun(@(c) c.DepartureTime - c.ArrivalTime, q.Served');
end
```

Option two: Use `cellfun` twice.

The outer call to `cellfun` means do something to each `ServiceQueue` object in `QSamples`. The "something" it does is to look at each customer in the `ServiceQueue` object's list `q.Served` and compute the time it spent in the system.

```
TimeInSystemSamples = cellfun( ...
    @(q) cellfun(@(c) c.DepartureTime - c.ArrivalTime, q.Served'), ...
    QSamples, ...
    UniformOutput=false);
```

Join them all into one big column.

```
TimeInSystem = vertcat(TimeInSystemSamples{:});
```

Now to calculate the mean waiting time in the system for our simulation:

```
WqSamples = cellfun( ...
    @(q) cellfun(@(c) c.BeginServiceTime - c.ArrivalTime, q.Served'), ...
    QSamples, ...
    UniformOutput=false);

Wq_all = vertcat(WqSamples{:});

Wq_sim = mean(Wq_all);
Wq_requirement = mean(Wq_all>5);
fprintf("Simulated W_q: %f\n", Wq_sim);
if Wq_requirement <= 0.1
    fprintf('s=%d meets the requirement.\n',s);
end
```

```
Simulated W_q: 91.762110
Simulated W_q: 68.482476
Simulated W_q: 47.620363
Simulated W_q: 27.871855
Simulated W_q: 10.648339
Simulated W_q: 3.656043
Simulated W_q: 0.965988
s=7 meets the requirement.
Simulated W_q: 0.445236
s=8 meets the requirement.
```

Pictures and stats for time customers spend in the system

Print out mean time spent in the system.

```
meanTimeInSystem = mean(TimeInSystem);
fprintf("Mean time in system: %f\n", meanTimeInSystem);
```

Make a figure with one set of axes.

```
fig = figure();
```

```
t = tiledlayout(fig,1,1);
ax = nexttile(t);
```

This time, the data is a list of real numbers, not integers. The option `BinWidth=...` means to use bins of a particular width, and choose the left-most and right-most edges automatically. Instead, you could specify the left-most and right-most edges explicitly. For instance, using `BinEdges=0:0.5:60` means to use bins $(0, 0.5), (0.5, 1.0), \ldots$

```
h = histogram(ax, TimeInSystem, Normalization="probability", BinWidth=5/60);
```

Add titles and labels and such.

```
title(ax, "Time in the system");
xlabel(ax, "Time");
ylabel(ax, "Probability");
```

Set ranges on the axes.

```
ylim(ax, [0, 0.2]);
xlim(ax, [0, 2.0]);
```

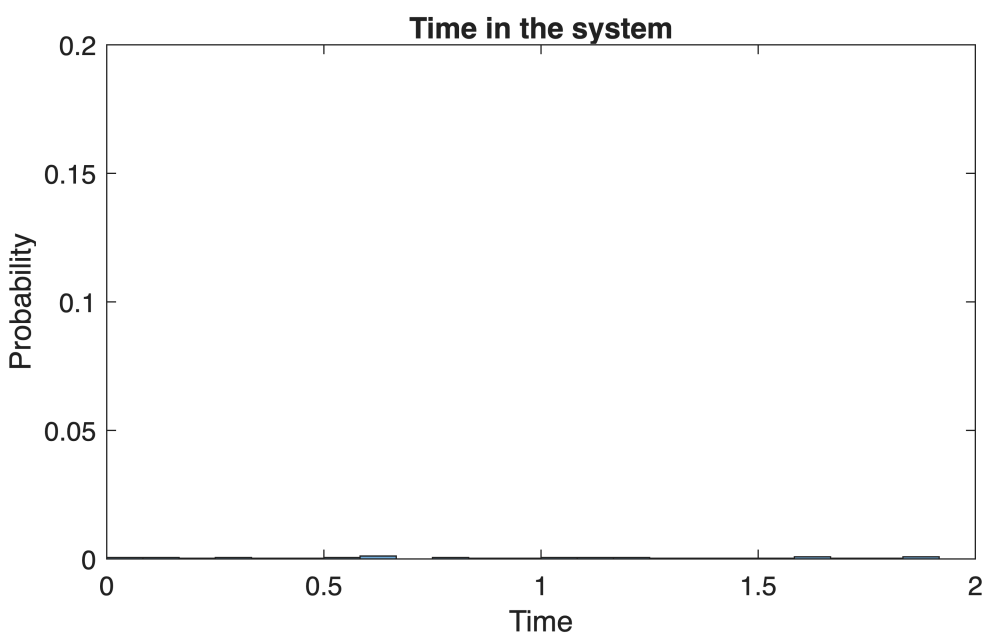
Wait for MATLAB to catch up.

```
pause(2);
```

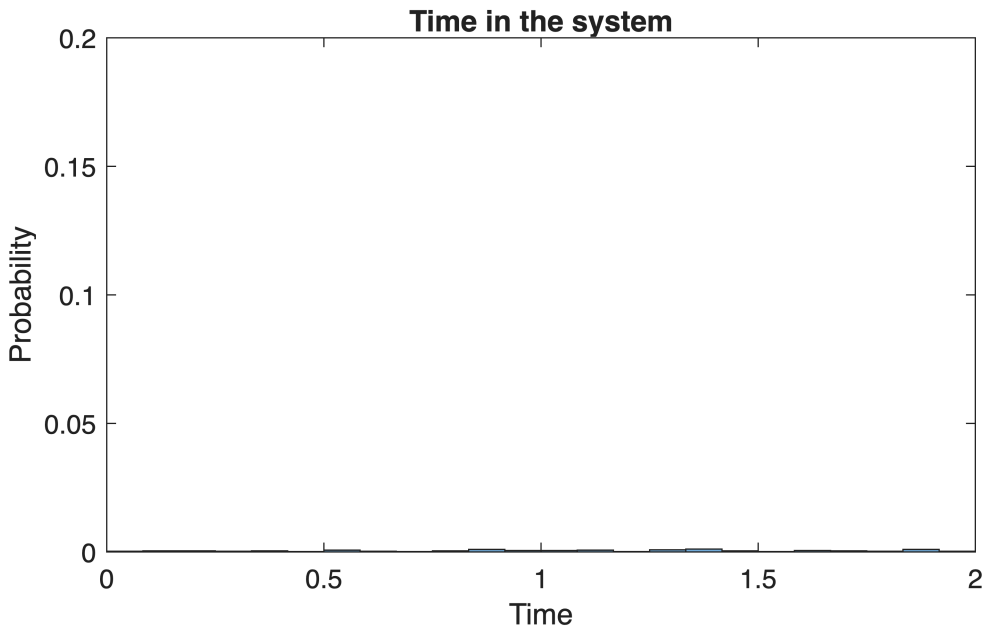
Save the picture as a PDF file.

```
exportgraphics(fig, "Time in system histogram.pdf");
end
```

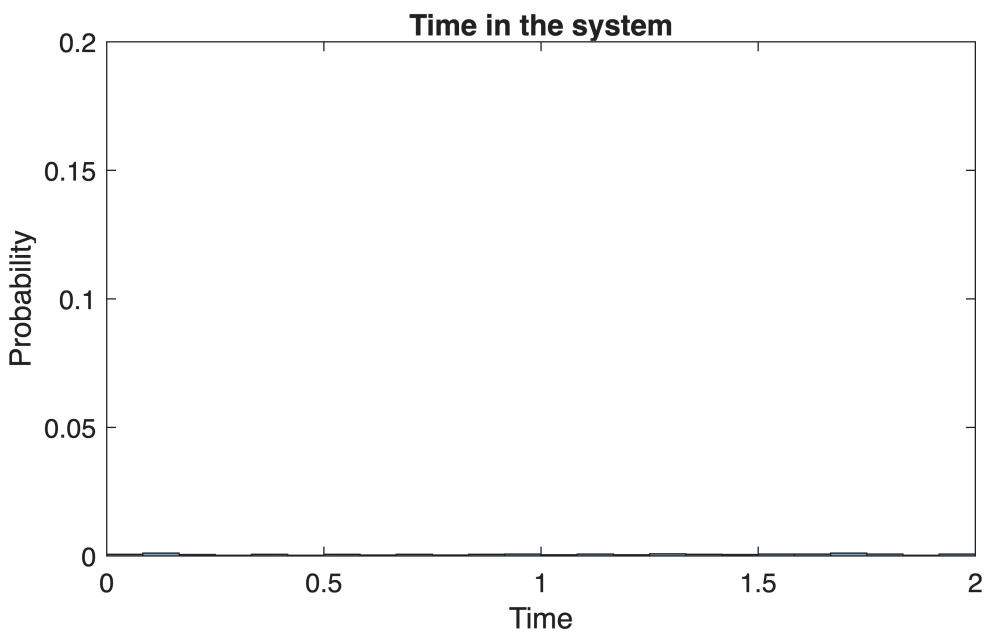
```
Mean time in system: 98.310680
```



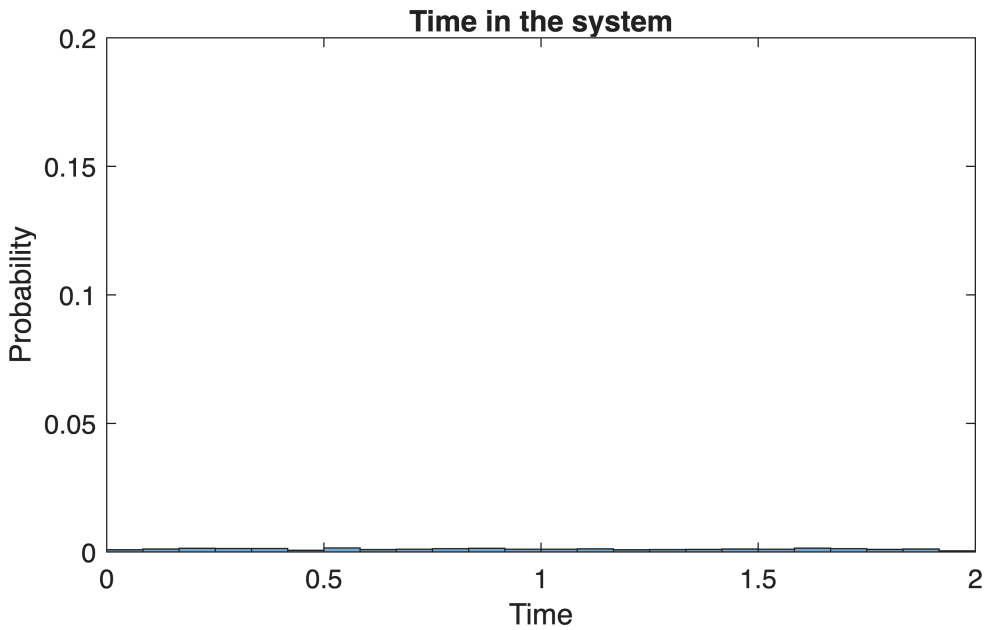

```
Mean time in system: 74.926276
```



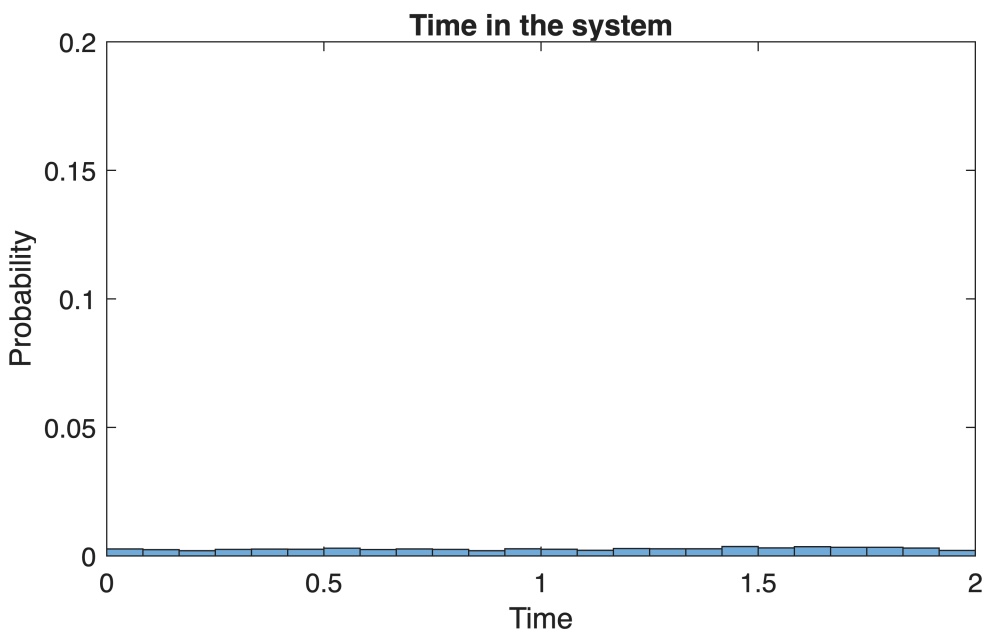

```
Mean time in system: 54.005502
```



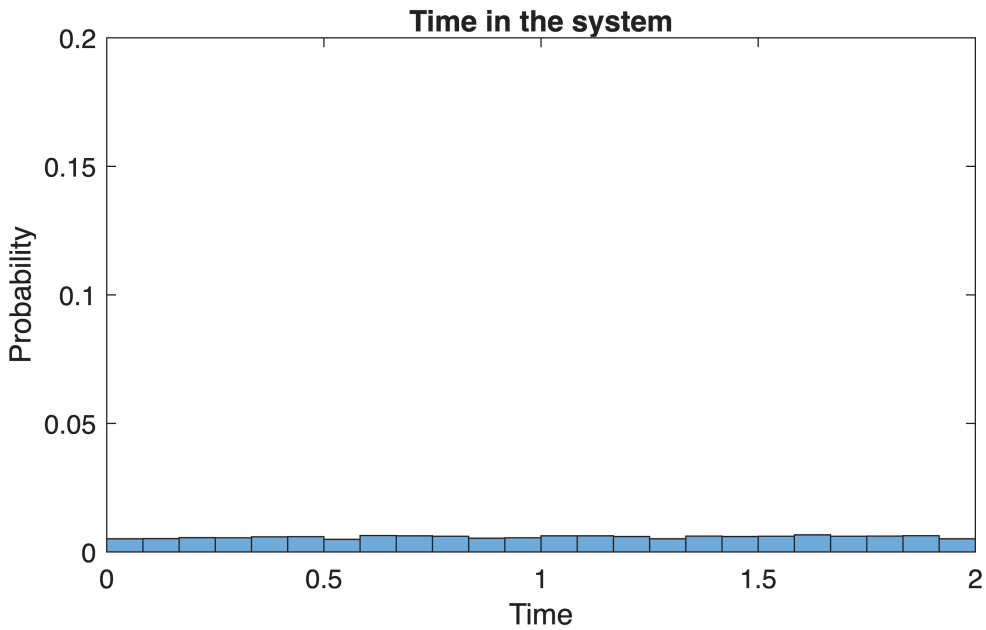

```
Mean time in system: 34.286184
```



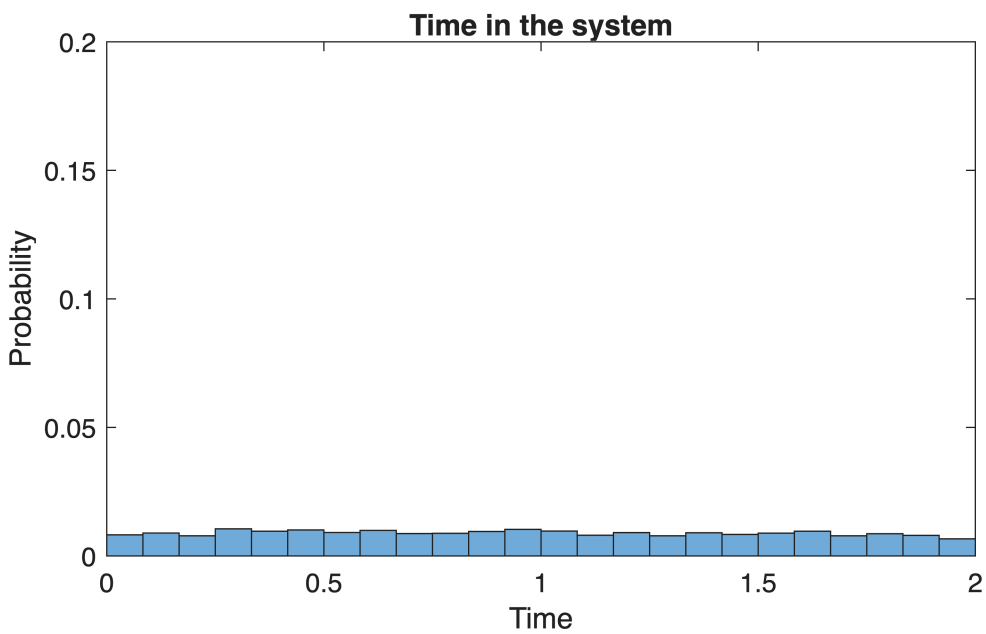

```
Mean time in system: 16.916317
```



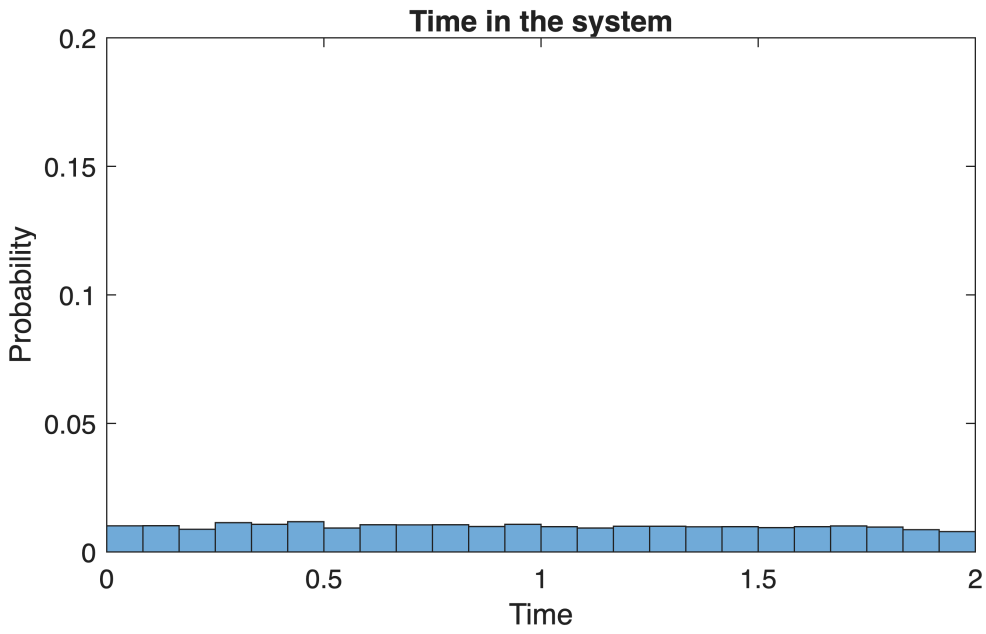

```
Mean time in system: 9.956934
```




```
Mean time in system: 7.218663
```




```
Mean time in system: 6.737097
```


Time in the system
Probability
0.2
0.15
0.1
0.05
0
0
0.5
1
1.5
2
Time